

CSIE 5452, Spring 2026: Homework 2

B13902022 Yu-Chi,Lai

1 Simulated Annealing for Priority Assignment

```
1  #include <stdio>
2  #include <fstream>
3  #include <cmath>
4  #include <chrono>
5  #include <cfloat>
6  #include <ctime>
7  #define MAX_LENGTH 50
8  #define TIME_LIMIT 15000000
9  const double INF = 1e100;
10
11 typedef struct msg {
12     int p; // priority
13     double tt; // transmission time;
14     double T; // period
15 } Msg;
16
17 double wrct(int i, int n, double tau, Msg m[]);
18
19 void print_info(int n, double tau, Msg m[]) {
20     printf("%d, %f\n", n, tau);
21     for (int i = 0; i < n; i++) {
22         printf("(%d %f %f): %f\n", m[i].p, m[i].tt, m[i].T, wrct(i, n, tau, m));
23     }
24 }
25
26 double wrct(int i, int n, double tau, Msg m[]) { // worst case response time
27     double Ci = m[i].tt;
28     double Bi = 0.0;
29     for (int k = 0; k < n; k++) {
30         if (m[k].p >= m[i].p) {
31             Bi = std::max(Bi, m[k].tt);
32         }
33     }
34
35     double Qi = Bi;
36     while (1) {
37         double temp = 0.0;
38         for (int k = 0; k < n; k++) {
39             if (m[k].p < m[i].p) {
40                 temp += std::ceil((Qi+tau)/m[k].T)*m[k].tt;
41             }
```

```

42     }
43     temp += Bi;
44
45     if (temp > m[i].T) {
46         return -1.0; // not schedulable
47     }
48
49     if (std::fabs(Qi-temp) < 1e-9) {
50         return Qi + Ci;
51     } else {
52         Qi = temp;
53     }
54 }
55 }
56
57 double sum_wcrct(int n, double tau, Msg m[]) {
58     double sum = 0.0;
59     for (int i = 0; i < n; i++) {
60         double Ri = wcrct(i, n, tau, m);
61         if (Ri < 0) return INF;
62         sum += Ri;
63     }
64     return sum;
65 }
66
67 int main() {
68     auto beg = std::chrono::high_resolution_clock::now();
69     srand(time(NULL));
70
71     int n;
72     double tau;
73     Msg m[MAX_LENGTH];
74     int bestP[MAX_LENGTH]; // best priority assignment
75     double minsum = DBL_MAX;
76
77     std::ifstream fin("input.dat");
78     fin >> n;
79     fin >> tau;
80
81     for (int i = 0; i < n; i++) {
82         fin >> m[i].p >> m[i].tt >> m[i].T;
83         bestP[i] = m[i].p;
84     }
85
86     double curr = sum_wcrct(n, tau, m);
87     if (curr < minsum) {
88         minsum = curr;
89         for (int i = 0; i < n; i++) bestP[i] = m[i].p;
90     }
91
92     double temperature = 5.0;

```

```
93     const double temp_min = 1e-6;
94     const double alpha = 0.997;
95     const int loops = 300;
96
97     while (temperature > temp_min) {
98         auto end = std::chrono::high_resolution_clock::now();
99         auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - beg);
100         if (duration.count() >= TIME_LIMIT) {
101             printf("%d ms passed", duration.count());
102             break;
103         }
104
105         for (int i = 0; i < loops; i++) {
106             int a = rand() % n, b = rand() % n;
107             while (a == b) b = rand() % n;
108             std::swap(m[a].p, m[b].p);
109
110             double cand = sum_wcrtn(n, tau, m);
111             double delta = cand - curr;
112
113             bool accept = false;
114             if (delta <= 0.0) accept = true;
115             else {
116                 double prob = std::exp(-delta / temperature);
117                 double x = (double) rand() / (RAND_MAX + 1.0);
118                 if (x <= prob) accept = true;
119             }
120
121             if (accept) {
122                 curr = cand;
123                 if (cand < minsum) {
124                     minsum = cand;
125                     for (int i = 0; i < n; i++) bestP[i] = m[i].p;
126                 }
127             } else {
128                 std::swap(m[a].p, m[b].p);
129             }
130         }
131
132         temperature *= alpha;
133     }
134
135     for (int i = 0; i < n; i++) {
136         printf("%d\n", bestP[i]);
137     }
138     printf("Optimized result: %f\n", minsum);
139
140 }
```

```
→ hw2 git:(main) ✗ ./a.out
14
6
0
7
3
5
11
2
9
10
1
16
4
8
15
13
12
Optimized result: 194.520000
```

Figure 1: The optimized result obtained from SA

2 Intersection Management: Part I

- 1.**
- 2.**

3 Intersection Management: Part II

4 Intersection Management: Part III

5 Realization of Level-X Autonomy

- 1.
- 2.
- 3.